LABORATORY MANUAL

VLSI Design Laboratory

Group B - Sequential Circuits

Tool Used: Xilinx Vivado Design Suite / ModelSim Simulator

Hardware Description Language: Verilog

(Any 3 of the 5 experiments listed are to be performed)

List of Experiments

Exp. No.	Title of Experiment
1	JK Flip-Flop and T Flip-Flop
2	3-bit Asynchronous (Ripple) Counter
3	3-bit Synchronous Up and Down Counter
4	Johnson Counter / Ring Counter
5	Serial-In Serial-Out (SISO) Shift Register

Experiment 1: JK Flip-Flop and T Flip-Flop

Aim

To design, simulate and verify the working of a JK flip-flop and a T flip-flop using Verilog HDL, and to observe their output responses on Vivado / ModelSim.

Learning Objective

To understand behavioral (RTL) modeling of edge-triggered sequential elements in Verilog, and to learn how to construct a self-checking testbench that generates a clock, applies stimulus, and monitors flip-flop outputs.

Learning Outcomes

- Write synthesizable Verilog code for JK and T flip-flops using always blocks.
- Construct a testbench with clock generation, reset, and stimulus sequencing.
- Simulate the design and interpret the resulting timing waveform.
- Correlate the simulated output with the standard JK/T flip-flop truth table.

Apparatus / Software

- Xilinx Vivado Design Suite / ModelSim Simulator
- PC with Windows / Linux Operating System
- Verilog HDL compiler (built into Vivado/ModelSim)

Theory

A JK flip-flop is a refinement of the SR flip-flop that eliminates the invalid state. It has two inputs, J and K. When J=K=0 the output holds its previous state; J=0,K=1 resets Q to 0; J=1,K=0 sets Q to 1; and J=K=1 toggles the output on every active clock edge. It is widely used in counters and control logic

because the toggle condition allows a single flip-flop to divide a clock frequency by two.

A T (Toggle) flip-flop is a simplified single-input version derived from the JK flip-flop by tying J and K together (T = J = K). When T=0 the output holds; when T=1 the output toggles on each clock edge. T flip-flops are the basic building block of ripple (asynchronous) and Johnson counters because of their natural divide-by-two behaviour.

Schematic

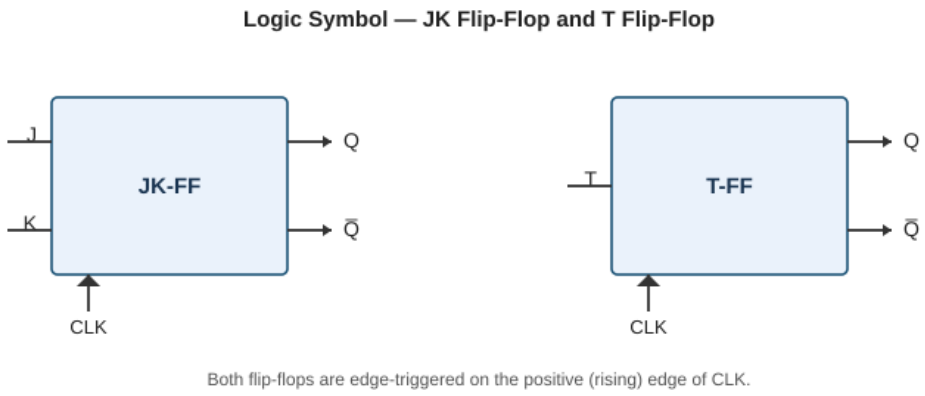


Fig. 1.1 - Block-level logic symbols of the JK flip-flop and T flip-flop

Layout

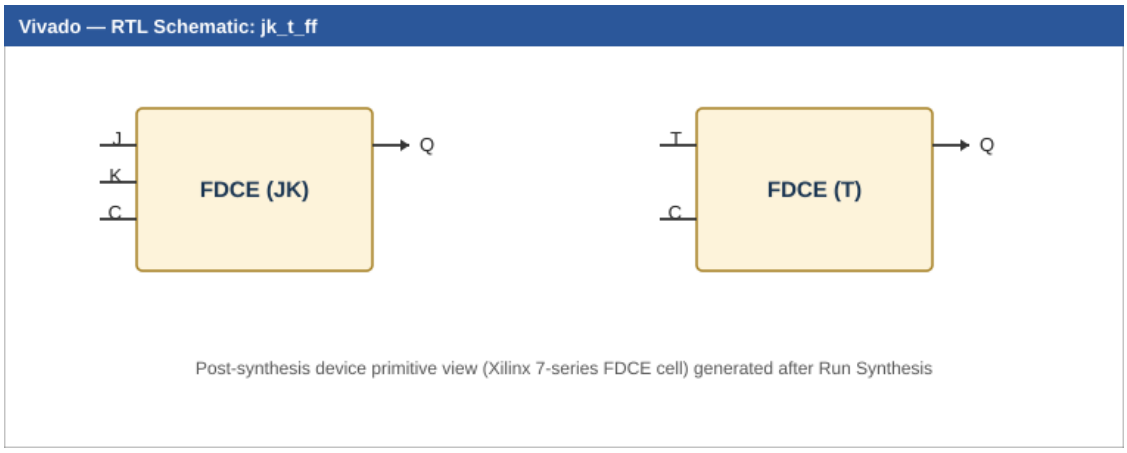


Fig. 1.2 - Post-synthesis RTL schematic (Vivado) showing FDCE primitives

Verilog Code

JK Flip-Flop:

```
module jk_ff (
    input  wire clk,
    input  wire rst,
    input  wire j,
    input  wire k,
    output reg  q,
    output wire qbar
);
    assign qbar = ~q;


    always @ (posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else begin
            case ({j, k})
                2'b00 : q <= q;      // No change  (hold)
                2'b01 : q <= 1'b0;   // Reset
                2'b10 : q <= 1'b1;   // Set
                2'b11 : q <= ~q;     // Toggle
            endcase
        end
    end
endmodule
```

Testbench - JK Flip-Flop:

```
module jk_ff_tb;
    reg  clk, rst, j, k;
    wire q, qbar;


    jk_ff uut (.clk(clk), .rst(rst), .j(j), .k(k), .q(q), .qbar(qbar));


    // 10 ns clock period
    initial clk = 0;
    always #5 clk = ~clk;


    initial begin
        rst = 1; j = 0; k = 0;
        #12 rst = 0;


        j = 0; k = 0;  #10;   // Hold
        j = 0; k = 1;  #10;   // Reset
        j = 1; k = 0;  #10;   // Set
        j = 1; k = 1;  #10;   // Toggle
        j = 1; k = 1;  #10;   // Toggle again
```

```
        $display("JK Flip-Flop simulation completed");
        $finish;
    end

    initial
        $monitor("t=%0t rst=%b j=%b k=%b -> q=%b qbar=%b",
                 $time, rst, j, k, q, qbar);
endmodule
```

T Flip-Flop:

```
module t_ff (
    input  wire clk,
    input  wire rst,
    input  wire t,
    output reg  q,
    output wire qbar
);
    assign qbar = ~q;

    always @ (posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else if (t)
            q <= ~q;     // Toggle
        else
            q <= q;      // Hold
    end
endmodule
```

Testbench - T Flip-Flop:

```
module t_ff_tb;
    reg  clk, rst, t;
    wire q, qbar;

    t_ff uut (.clk(clk), .rst(rst), .t(t), .q(q), .qbar(qbar));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        rst = 1; t = 0;
```

```
        #12 rst = 0;


        t = 1; #10;   // Toggle
        t = 1; #10;   // Toggle
        t = 0; #10;   // Hold
        t = 0; #10;   // Hold
        t = 1; #10;   // Toggle


        $display("T Flip-Flop simulation completed");
        $finish;
    end


    initial
        $monitor("t=%0t rst=%b t=%b -> q=%b qbar=%b",
                 $time, rst, t, q, qbar);
endmodule
```

Output Waveform

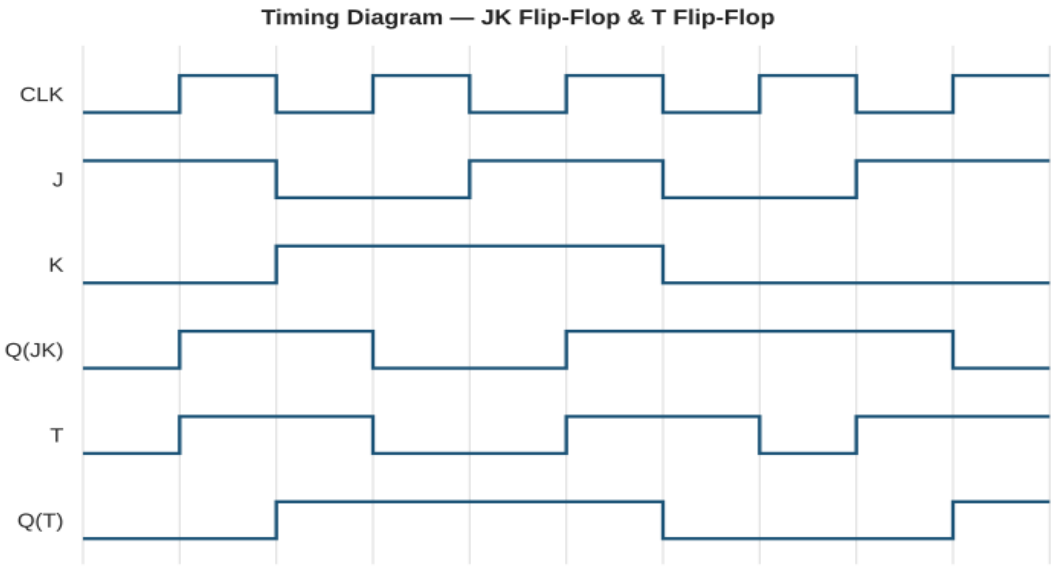


Fig. 1.3 - Simulated timing diagram for JK-FF and T-FF

Results

Table 1.1 - JK Flip-Flop Truth Table Verification

Clock Edge	J	K	Q (before)	Q (after)	Operation
1	0	0	0	0	Hold
2	0	1	0	0	Reset
3	1	0	0	1	Set
4	1	1	1	0	Toggle
5	1	1	0	1	Toggle

Table 1.2 - T Flip-Flop Truth Table Verification

Clock Edge	T	Q (before)	Q (after)	Operation
1	1	0	1	Toggle
2	1	1	0	Toggle
3	0	0	0	Hold
4	0	0	0	Hold
5	1	0	1	Toggle

Conclusion

The JK flip-flop and T flip-flop were successfully designed in Verilog HDL and verified through simulation. The simulated waveforms matched the expected truth-table behaviour for hold, set, reset and toggle conditions, confirming correct edge-triggered operation. The T flip-flop's toggle property was further verified as the building block used in subsequent ripple/Johnson counter experiments.

Experiment 2: 3-bit Asynchronous (Ripple) Counter

Aim

To design, simulate and verify a 3-bit asynchronous (ripple) up-counter using Verilog HDL.

Learning Objective

To understand how cascading toggle flip-flops, each clocked by the previous stage's output, produces an asynchronous ripple counter, and to observe the propagation delay this introduces between stages.

Learning Outcomes

- Model a ripple counter in Verilog by cascading multiple always blocks.
- Differentiate between synchronous and asynchronous clocking schemes.
- Observe and explain ripple/propagation delay in simulation waveforms.
- Verify the binary count sequence 000 through 111 and rollover.

Apparatus / Software

- Xilinx Vivado Design Suite / ModelSim Simulator
- PC with Windows / Linux Operating System

Theory

An asynchronous (ripple) counter is built from toggle flip-flops connected in cascade, where each flip-flop is clocked not by the system clock but by the output of the preceding flip-flop. Only the first flip-flop receives the external clock; Q0 drives the clock of FF1, and Q1 drives the clock of FF2. Each stage divides the frequency of the previous stage by two, so the three flip-flops together count from 000 to 111 and roll over. Because each stage's transition depends on the previous one toggling, the change 'ripples' through the chain,

introducing a small cumulative propagation delay - the key disadvantage compared with synchronous counters.

Schematic

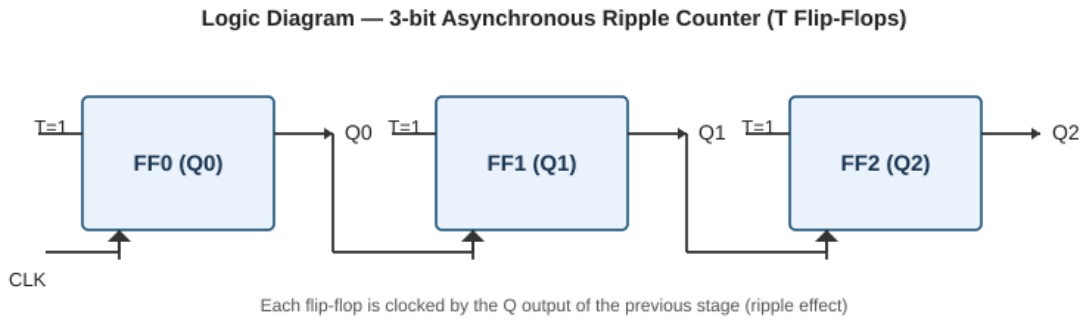


Fig. 2.1 - Logic diagram of the 3-bit ripple counter (T flip-flops in cascade)

Layout

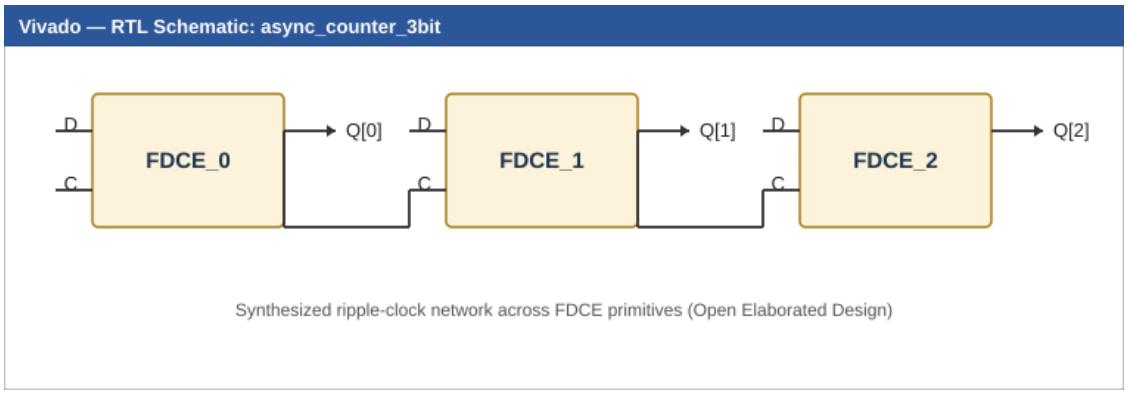


Fig. 2.2 - Post-synthesis RTL schematic showing the ripple clock network

Verilog Code

```
module async_counter_3bit (
    input  wire clk,
    input  wire rst,
    output wire [2:0] q
);
    reg q0, q1, q2;


    // Stage 0 - toggles on every rising edge of the external clock
    always @ (posedge clk or posedge rst) begin
        if (rst) q0 <= 1'b0;
        else     q0 <= ~q0;
    end
```

```
    // Stage 1 - clocked by Q0 (ripple connection)
    always @ (posedge q0 or posedge rst) begin
        if (rst) q1 <= 1'b0;
        else     q1 <= ~q1;
    end

    // Stage 2 - clocked by Q1 (ripple connection)
    always @ (posedge q1 or posedge rst) begin
        if (rst) q2 <= 1'b0;
        else     q2 <= ~q2;
    end

    assign q = {q2, q1, q0};
endmodule
```

Testbench:

```
module async_counter_3bit_tb;
    reg  clk, rst;
    wire [2:0] q;

    async_counter_3bit uut (.clk(clk), .rst(rst), .q(q));

    initial clk = 0;
    always #5 clk = ~clk;     // 10 ns clock period

    initial begin
        rst = 1;
        #12 rst = 0;
        #160;                 // run for 16 clock cycles
        $display("3-bit Asynchronous Counter simulation completed");
        $finish;
    end

    initial
        $monitor("t=%0t rst=%b q=%b (%0d)", $time, rst, q, q);
endmodule
```

Output Waveform

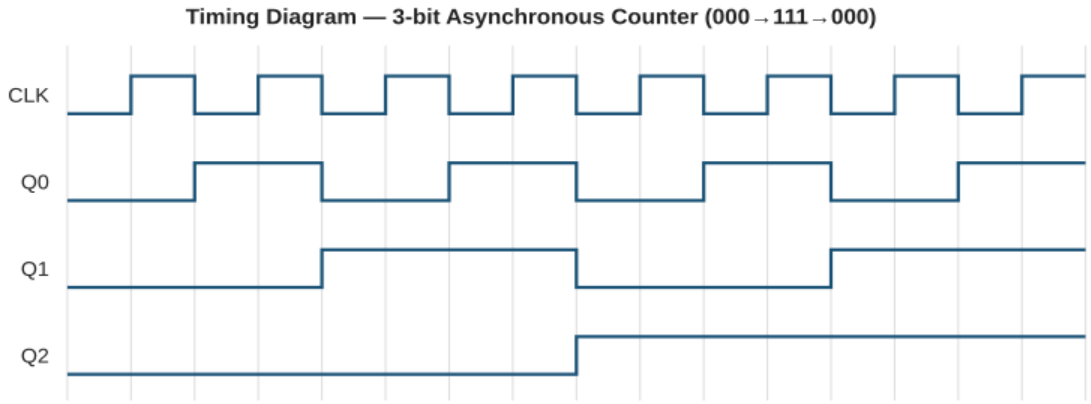


Fig. 2.3 - Simulated waveform showing ripple count sequence 000→111→000

Results

Table 2.1 - Count Sequence Verification

Clock Pulse #	Q2	Q1	Q0	Decimal Value
0	0	0	0	0
1	0	0	1	1
2	0	1	0	2
3	0	1	1	3
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6
7	1	1	1	7
8	0	0	0	0 (rollover)

Conclusion

The 3-bit asynchronous ripple counter was successfully implemented and simulated. The output sequence correctly progressed from 000 to 111 and rolled over to 000, confirming correct cascaded toggling. The simulation also illustrated the inherent ripple delay between stages, highlighting why asynchronous counters are unsuitable for high-speed applications compared to synchronous counters.

Experiment 3: 3-bit Synchronous Up and Down Counter

Aim

To design, simulate and verify a 3-bit synchronous up/down counter using Verilog HDL, controlled by a mode-select input.

Learning Objective

To understand synchronous counter design where all flip-flops share a common clock, and to implement a direction-controlled (up/down) counter using behavioral arithmetic operators in Verilog.

Learning Outcomes

- Design a common-clock (synchronous) counter as opposed to a ripple counter.
- Use a control signal to switch counting direction at run time.
- Verify both up-counting and down-counting sequences through simulation.
- Compare synchronous counter timing behaviour with the asynchronous counter of Experiment 2.

Apparatus / Software

- Xilinx Vivado Design Suite / ModelSim Simulator
- PC with Windows / Linux Operating System

Theory

In a synchronous counter, every flip-flop is triggered by the same system clock simultaneously, eliminating the cumulative propagation delay seen in ripple counters. A 3-bit up/down counter uses a control input (UP_DOWN) to decide the next state: when UP_DOWN=1 the register increments (q+1) on every clock edge, producing the sequence 000→001→...→111→000; when UP_DOWN=0

it decrements (q−1), producing 111→110→...→000→111. Because the next state of all bits is computed and updated together on the same clock edge, all outputs change simultaneously, giving glitch-free, predictable timing - the principal advantage of synchronous design.

Schematic

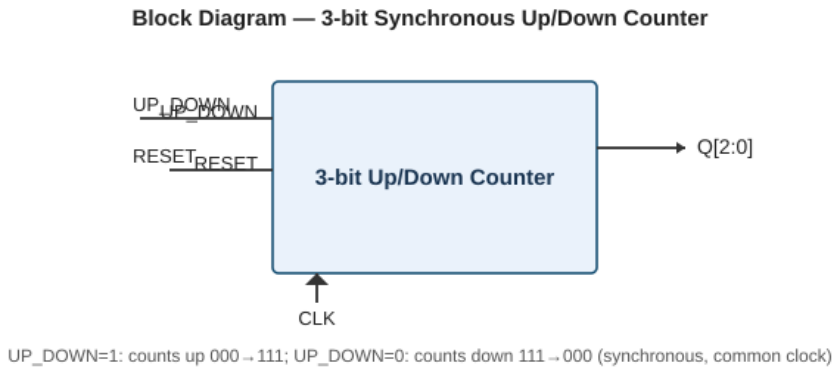


Fig. 3.1 - Block diagram of the 3-bit synchronous up/down counter

Layout

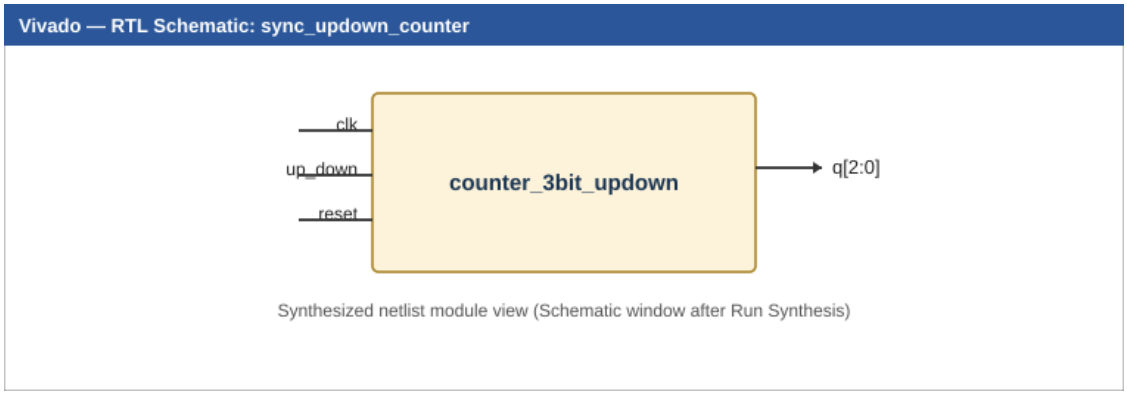


Fig. 3.2 - Post-synthesis RTL schematic module view

Verilog Code

```
module sync_updown_counter (
    input  wire clk,
    input  wire rst,
    input  wire up_down,    // 1 = count up, 0 = count down
    output reg  [2:0] q
);
    always @ (posedge clk or posedge rst) begin
        if (rst)
```

```
        q <= 3'b000;
    else if (up_down)
        q <= q + 1'b1;   // synchronous increment
    else
        q <= q - 1'b1;   // synchronous decrement
  end
endmodule
```

Testbench:

```
module sync_updown_counter_tb;
  reg  clk, rst, up_down;
  wire [2:0] q;

  sync_updown_counter uut (.clk(clk), .rst(rst), .up_down(up_down), .q(q));

  initial clk = 0;
  always #5 clk = ~clk;       // 10 ns clock period

  initial begin
    rst = 1; up_down = 1;
    #12 rst = 0;

    up_down = 1;  #80;       // count UP for 8 clocks  : 000 -> 111
    up_down = 0;  #80;       // count DOWN for 8 clocks: 111 -> 000

    $display("3-bit Synchronous Up/Down Counter simulation completed");
    $finish;
  end

  initial
    $monitor("t=%0t rst=%b up_down=%b q=%b (%0d)", $time, rst, up_down, q, q);
endmodule
```

Output Waveform

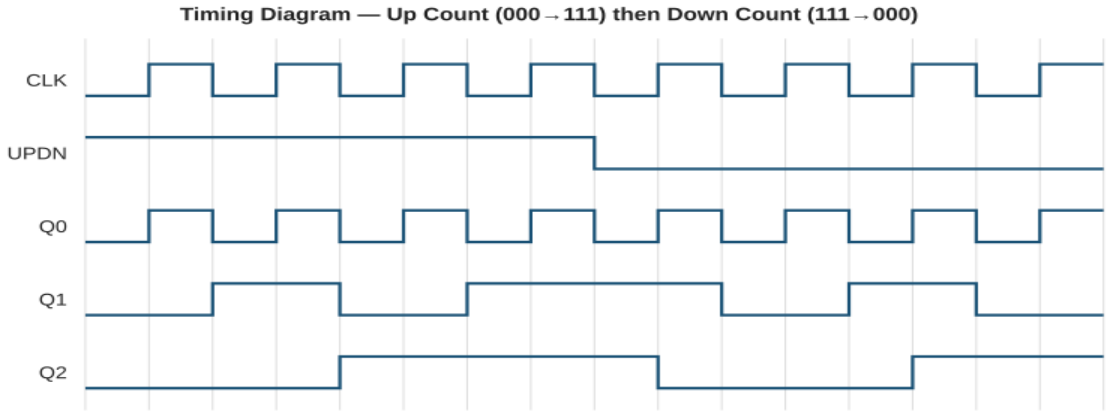


Fig. 3.3 - Simulated waveform: up-count 000→111 followed by down-count 111→000

Results

Table 3.1 - Up/Down Count Sequence Verification

Clock Pulse #	UP_DOWN	Q2 Q1 Q0	Decimal Value	Mode
1	1	001	1	Up
2	1	010	2	Up
3	1	011	3	Up
4	1	111	7	Up (final)
5	0	110	6	Down
6	0	101	5	Down
7	0	100	4	Down
8	0	000	0	Down (final)

Conclusion

The 3-bit synchronous up/down counter was implemented and verified successfully. With UP_DOWN=1 the counter incremented correctly from 000 to 111, and with UP_DOWN=0 it decremented correctly back to 000, with all bits transitioning simultaneously on the common clock edge. This confirmed the absence of ripple delay and validated the direction-control logic.

Experiment 4: Johnson Counter / Ring Counter

Aim

To design, simulate and verify a 4-bit Johnson counter and a 4-bit Ring counter using Verilog HDL.

Learning Objective

To understand how a shift register with feedback can be configured to generate cyclic count sequences, and to distinguish the inverted-feedback (Johnson) configuration from the direct-feedback (Ring) configuration.

Learning Outcomes

- Model feedback shift registers in Verilog using concatenation operators.
- Differentiate Johnson and Ring counter feedback connections and resulting sequences.
- Verify the unique 2N-state Johnson sequence and the N-state Ring sequence through simulation.
- Appreciate applications of these counters as sequence/timing generators.

Apparatus / Software

- Xilinx Vivado Design Suite / ModelSim Simulator
- PC with Windows / Linux Operating System

Theory

A Johnson counter (twisted-ring counter) is a shift register in which the complement of the last flip-flop's output is fed back to the first flip-flop's input. For an n-bit register this produces a unique sequence of 2n states before repeating - for n=4 the counter cycles through 8 distinct codes (0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001) and back to 0000. A Ring counter instead feeds the last output back directly (uncomplemented), so a single '1' (or '0')

simply circulates around the register, giving only n distinct states for n flip-flops. Both circuits require only a shift register and one feedback path, making them simple, glitch-free sequence generators commonly used for control/timing-state machines.

Schematic

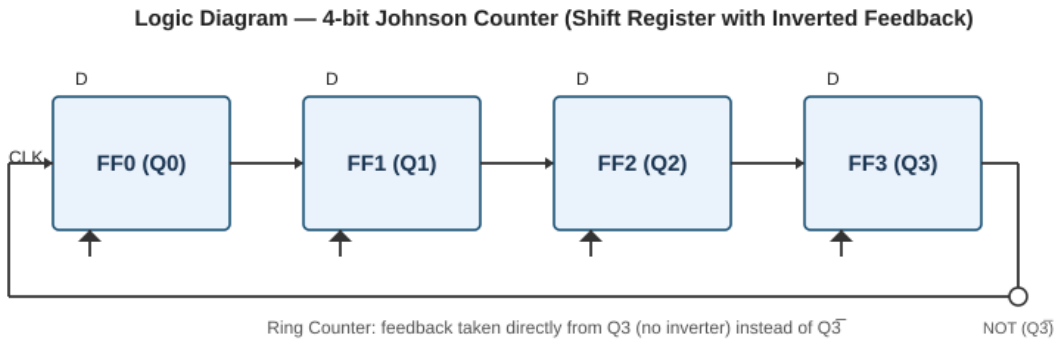


Fig. 4.1 - 4-bit shift register with inverted feedback (Johnson) / direct feedback (Ring)

Layout

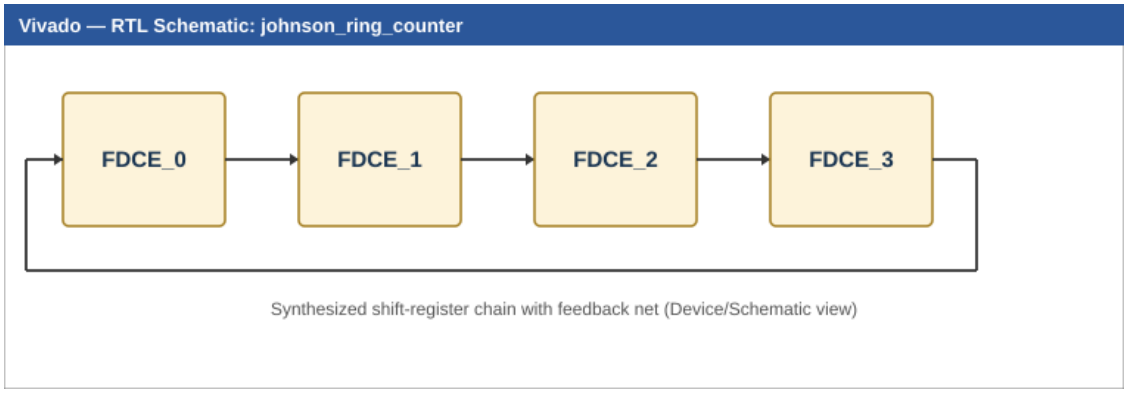


Fig. 4.2 - Post-synthesis RTL schematic of the feedback shift chain

Verilog Code

Johnson Counter:

```
module johnson_counter_4bit (
    input  wire clk,
    input  wire rst,
    output reg  [3:0] q
);
```

```
    always @ (posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {~q[0], q[3:1]};   // shift right, inverted feedback
    end
endmodule
```

Ring Counter:

```
module ring_counter_4bit (
    input  wire clk,
    input  wire rst,
    output reg  [3:0] q
);
    always @ (posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0001;           // preset a single '1'
        else
            q <= {q[0], q[3:1]};     // shift right, direct feedback
    end
endmodule
```

Testbench:

```
module johnson_ring_counter_tb;
    reg  clk, rst;
    wire [3:0] q_johnson, q_ring;


    johnson_counter_4bit uut1 (.clk(clk), .rst(rst), .q(q_johnson));
    ring_counter_4bit    uut2 (.clk(clk), .rst(rst), .q(q_ring));


    initial clk = 0;
    always #5 clk = ~clk;        // 10 ns clock period


    initial begin
        rst = 1;
        #12 rst = 0;
        #160;                    // observe 16 clock cycles (2 full Johnson sequences)
        $display("Johnson / Ring Counter simulation completed");
        $finish;
    end
```

```
    initial
        $monitor("t=%0t rst=%b johnson_q=%b ring_q=%b", $time, rst, q_johnson, q_ring);
endmodule
```

Output Waveform

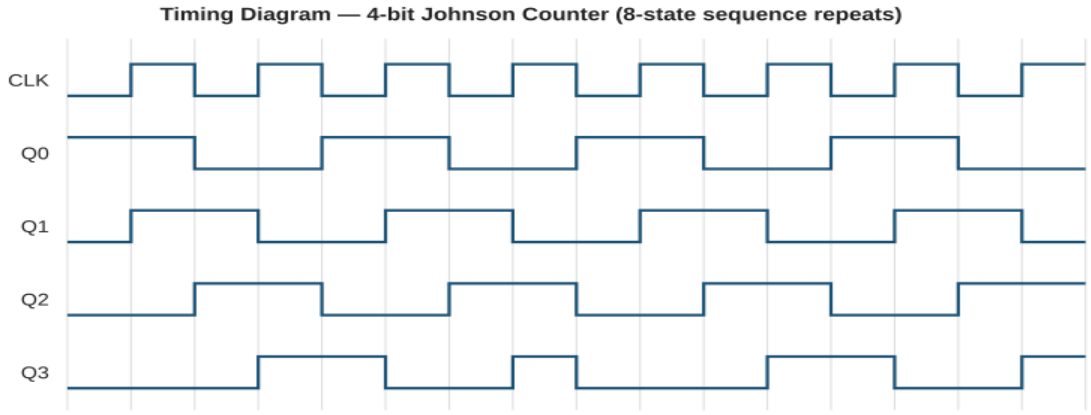


Fig. 4.3 - Simulated waveform of the 4-bit Johnson counter (8-state repeating sequence)

Results

Table 4.1 - Johnson Counter Sequence (4-bit, 8 states)

Clock Pulse #	Q3	Q2	Q1	Q0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	1
3	0	1	1	1
4	1	1	1	1
5	1	1	1	0
6	1	1	0	0
7	1	0	0	0
8	0	0	0	0 (repeats)

Conclusion

Both the Johnson and Ring counters were implemented and simulated successfully. The Johnson counter produced the expected unique 8-state cyclic sequence using inverted feedback, while the Ring counter circulated a single active bit through all 4 states using direct feedback. The results confirmed the theoretical state sequences and demonstrated the simplicity of feedback shift registers as cyclic sequence generators.

Experiment 5: Serial-In Serial-Out (SISO) Shift Register

Aim

To design, simulate and verify a 4-bit Serial-In Serial-Out (SISO) shift register using Verilog HDL.

Learning Objective

To understand the construction of a shift register from cascaded flip-flops and to observe how serial data introduced at one end propagates through the register, appearing at the output after a fixed number of clock cycles.

Learning Outcomes

- Model a shift register in Verilog using the concatenation/shift operator.
- Understand serial data transfer and the resulting latency through an n-bit register.
- Verify, through simulation, that input data reappears at the output after n clock cycles.
- Relate SISO registers to serial communication and data buffering applications.

Apparatus / Software

- Xilinx Vivado Design Suite / ModelSim Simulator
- PC with Windows / Linux Operating System

Theory

A Serial-In Serial-Out (SISO) shift register consists of a chain of flip-flops in which data enters serially at the input of the first flip-flop and is shifted one position on every clock pulse; the output is taken serially from the last flip-flop. For a 4-bit register, a bit applied at SI appears at SO exactly 4 clock pulses later,

since it must propagate through all four stages. Internally this is modeled as a 4-bit register where, on each clock edge, the new serial input is loaded into the most-significant position while existing bits shift one place towards the least-significant (output) position. SISO registers are fundamental to serial communication interfaces, time-delay elements, and serial-to-parallel data conversion circuits.

Schematic

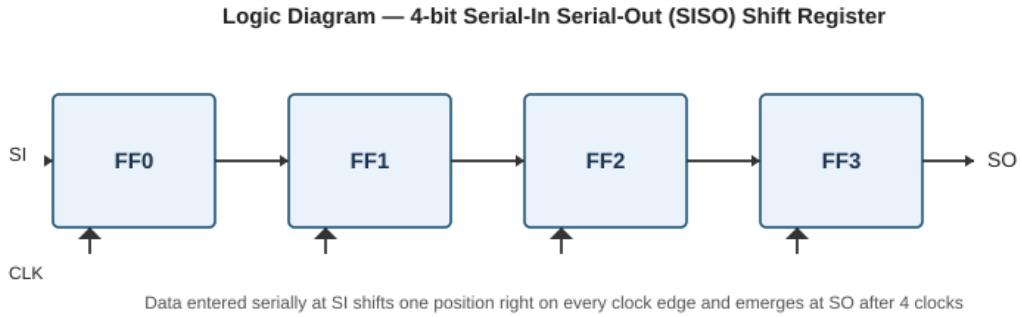


Fig. 5.1 - Logic diagram of the 4-bit SISO shift register

Layout

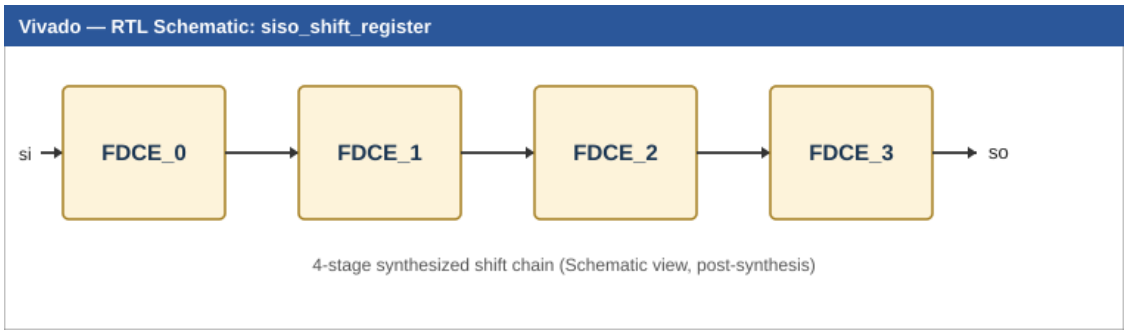


Fig. 5.2 - Post-synthesis RTL schematic of the 4-stage shift chain

Verilog Code

```
module siso_shift_register (
    input  wire clk,
    input  wire rst,
    input  wire si,        // serial input
    output wire so         // serial output
);
    reg [3:0] shreg;
```

```
    always @ (posedge clk or posedge rst) begin
        if (rst)
            shreg <= 4'b0000;
        else
            shreg <= {si, shreg[3:1]};   // shift right, load si at MSB
    end


    assign so = shreg[0];   // serial output taken from LSB
endmodule
```

Testbench:

```
module siso_shift_register_tb;
    reg  clk, rst, si;
    wire so;


    siso_shift_register uut (.clk(clk), .rst(rst), .si(si), .so(so));


    initial clk = 0;
    always #5 clk = ~clk;          // 10 ns clock period


    initial begin
        rst = 1; si = 0;
        #12 rst = 0;


        // serial data stream applied at SI, one bit per clock
        si = 1; #10;
        si = 1; #10;
        si = 0; #10;
        si = 0; #10;
        si = 1; #10;
        si = 1; #10;
        si = 1; #10;
        si = 1; #10;


        $display("SISO Shift Register simulation completed");
        $finish;
    end


    initial
        $monitor("t=%0t rst=%b si=%b so=%b", $time, rst, si, so);
endmodule
```

Output Waveform

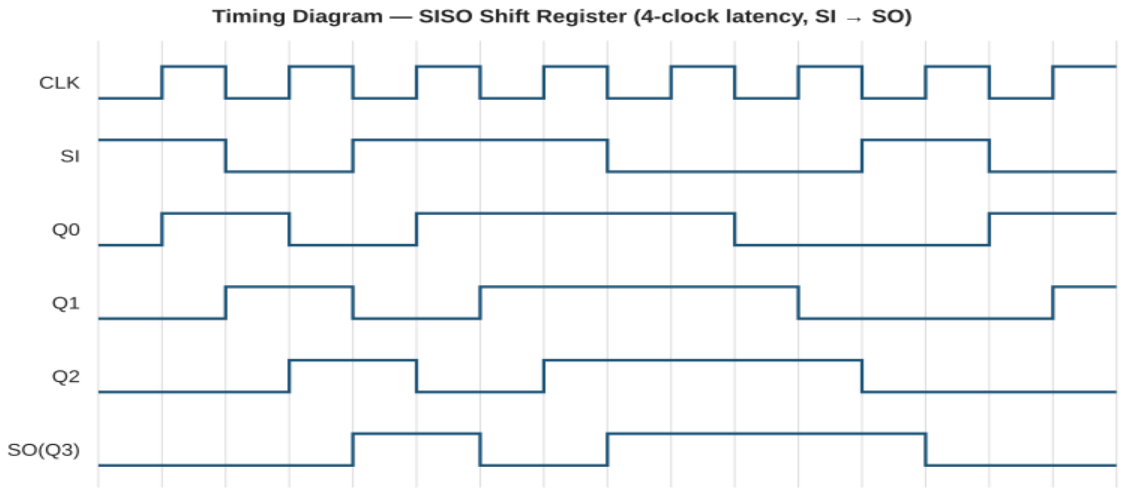


Fig. 5.3 - Simulated waveform showing 4-clock latency between SI and SO

Results

Table 5.1 - Serial Data Propagation Verification

Clock Pulse #	SI (input)	Shift Register [Q3 Q2 Q1 Q0]	SO (output)
1	1	1000	0
2	1	1100	0
3	0	0110	0
4	0	0011	1 (= SI at pulse 1)
5	1	1001	1 (= SI at pulse 2)
6	1	1100	0 (= SI at pulse 3)
7	1	1110	0 (= SI at pulse 4)
8	1	1111	1 (= SI at pulse 5)

Conclusion

The 4-bit SISO shift register was successfully designed and verified in simulation. The bit pattern applied at the serial input correctly reappeared at the serial output after a delay of exactly four clock cycles, confirming correct shifting operation and validating the register as a serial delay/buffer element.